FCyT
Sede Concepción del Uruguay

Licenciatura en Sistemas de Información

Programación Moviles - 2026

TRABAJO PRÁCTICO

N° 7

Satellite Tracker

Visualización de geoposiciones satelitales en tiempo real y predictor de visibilidad con propagación orbital SGP4

Área temática: Redes de Datos, Telecomunicaciones y Telecomunicaciones Satelitales

Tecnología: React Native

Nivel: Medio-Avanzado

FCyT
Sede Concepción del Uruguay

Licenciatura en Sistemas de Información

Programación Moviles - 2026

1. Introducción y contexto

Este trabajo práctico propone el desarrollo de una aplicación móvil en React Native que calcule, en tiempo real, la posición de satélites artificiales (órbitas LEO, MEO y GEO) a partir de datos orbitales reales obtenidos de APIs públicas, la muestre como geoposición sobre un mapa mundial y determine cuándo un satélite específico será visible desde la ubicación del usuario.

El proyecto introduce al estudiante en conceptos de mecánica orbital aplicada a las telecomunicaciones satelitales: propagación de órbitas, cálculo del punto subsatelital (latitud, longitud, altitud), azimut/elevación, ventanas de visibilidad y su relación directa con el diseño de enlaces satelitales (por ejemplo, para backhaul de estaciones remotas o terminales VSAT). A diferencia de una app de mapas convencional, aquí la lógica de negocio central es matemática y astronómica, lo que eleva sustancialmente la complejidad respecto de un CRUD típico.

Como caso de estudio de telecomunicaciones satelitales se incluye la constelación Starlink. Starlink no ofrece una API pública para seguimiento de sus satélites (su Space Traffic Coordination API está restringida a operadores satelitales y requiere un certificado mTLS firmado por SpaceX), por lo que el TP utiliza los datos orbitales suplementarios derivados de las efemérides que SpaceX publica y que distribuye CelesTrak. La app debe tratar a estas fuentes como proveedores intercambiables detrás de una capa de datos propia.

2. Objetivos

2.1 Objetivo general

Implementar una aplicación móvil que obtenga datos orbitales de APIs públicas, calcule localmente la posición de satélites mediante modelos de propagación estándar de la industria (SGP4/SDP4), la represente como geoposición sobre un mapa (marcador, trayectoria y área de cobertura) y prediga la visibilidad de los satélites desde una ubicación geográfica dada, con notificaciones de pase.

2.2 Objetivos específicos

- Consumir y parsear datos orbitales desde fuentes públicas (CelesTrak, Space-Track, N2YO).
- Propagar la órbita de un satélite en el tiempo utilizando el algoritmo SGP4.
- Calcular azimut y elevación del satélite respecto de la posición GPS del usuario.
- Determinar ventanas de visibilidad (AOS/LOS — Acquisition/Loss of Signal) para un satélite dado.
- Visualizar la trayectoria del pase en un gráfico polar (sky chart).
- Notificar al usuario minutos antes de un pase visible relevante.
- Ofrecer un modo de orientación asistida (brújula) para apuntar hacia el satélite.

- Calcular el punto subsatelital (latitud, longitud, altitud) y mostrarlo en tiempo real sobre un mapa mundial.
- Dibujar el ground track (trayectoria sobre la superficie terrestre) y el footprint (área de cobertura) del satélite seleccionado.
- Mostrar una capa de constelación (por ejemplo, un subconjunto de Starlink) con múltiples satélites simultáneos sin degradar el rendimiento de la UI.
- Diseñar una capa de acceso a datos con múltiples proveedores, caché local, respeto de los límites de uso de cada API y conmutación ante fallas.
- Validar la exactitud de las posiciones calculadas comparándolas contra una fuente independiente (N2YO y Where the ISS at?) y documentar el error observado.

3. Requisitos funcionales

ID	Requisito	Prioridad
RF-01	Buscar y seleccionar satélites por nombre o NORAD ID desde el catálogo obtenido de CelesTrak/Space-Track	Alta
RF-02	Calcular posición actual (lat/lon/alt) del satélite seleccionado mediante SGP4 local	Alta
RF-03	Calcular azimut/elevación del satélite respecto a la ubicación del usuario	Alta
RF-04	Determinar próximas ventanas de visibilidad (AOS, punto máximo, LOS)	Alta
RF-05	Graficar la trayectoria del pase en un sky chart polar (SVG)	Alta
RF-06	Notificar al usuario N minutos antes de un pase configurado	Media
RF-07	Modo brújula: overlay indicando hacia dónde apuntar el dispositivo	Media
RF-08	Filtrar catálogo por constelación (Starlink, ISS, GPS, meteorológicos, etc.)	Media
RF-09	Actualizar automáticamente los datos orbitales cuando superen su TTL (12-24 h), sin exceder los límites de cada proveedor	Media
RF-10	Mapa mundial con la posición en tiempo real del satélite seleccionado (marcador animado, refresco cada 1-5 s calculado localmente)	Alta
RF-11	Ground track: trayectoria pasada y futura (al menos una órbita completa) dibujada sobre el mapa, resolviendo el cruce del antimeridiano	Alta
RF-12	Footprint: área de cobertura del satélite según su altitud y una elevación mínima configurable	Media
RF-13	Capa de constelación: mostrar múltiples satélites (p. ej. subconjunto de Starlink) con límite de objetos o agrupamiento	Media
RF-14	Abstracción de proveedores de datos con caché local y fallback automático entre fuentes	Alta

Licenciatura en Sistemas de Información

Programación Moviles - 2026

ID	Requisito	Prioridad
RF-15	Modo validación: comparar la posición calculada contra N2YO (cualquier satélite) y Where the ISS at? (ISS) y mostrar el error en km	Media
RF-16	Indicar al usuario la antigüedad de los datos orbitales usados y alertar cuando estén desactualizados	Baja

4. Requisitos técnicos y stack sugerido

4.1 Cálculo orbital

- satellite.js — implementación JS de SGP4/SDP4, propagación de posición ECI y conversión a lat/lon/alt (geodético).
- Conversión de coordenadas ECI → topocéntricas (azimut/elevación) respecto del observador.
- Soporte de formatos OMM/JSON además de TLE: los objetos nuevos con número de catálogo de 6 dígitos no tienen representación TLE, por lo que se recomienda usar JSON/OMM como formato principal (verificar el soporte de la versión de satellite.js utilizada).

4.2 Fuentes de datos (APIs públicas)

Proveedor	Uso en el TP	Acceso y límites
CelesTrak	Fuente principal de datos orbitales: catálogo GP y sets suplementarios (SupGP) de Starlink derivados de efemérides del operador, vía sup-gp.php?FILE=starlink	Sin API key. Revisa sus fuentes una vez por día: TTL de caché de 12-24 h
Space-Track.org	Catálogo oficial GP/TLE e históricos; fuente alternativa/fallback	Requiere cuenta (identity/password). Guía: menos de 30 requests/min y 300/hora
N2YO	Validación cruzada: TLE, posiciones con az/el, pases visuales y de radio (hasta 10 días), objetos sobre el observador	API key gratuita. 1.000 transacciones/hora (TLE, posiciones); 100/hora (pases, above)

Licenciatura en Sistemas de Información

Programación Moviles - 2026

Proveedor	Uso en el TP	Acceso y límites
Where the ISS at?	Prototipo rápido y validación de la ISS (posición, altitud, velocidad, TLE)	Sin autenticación, aprox. 1 request/s. Solo ISS (NORAD 25544)

Alcance respecto de Starlink: no existe una API pública oficial para consultar posiciones de sus satélites. Las APIs Enterprise de Starlink (cuentas, terminales, routers, telemetría) requieren una cuenta business y no exponen la constelación, y la Space Traffic Coordination API está restringida a operadores satelitales. El trabajo debe documentar esta limitación y justificar el uso del set suplementario.

4.3 Estrategia de acceso a datos y límites de uso

- Las posiciones se calculan localmente con SGP4: las APIs se consultan solo para obtener datos orbitales (cada 12-24 h) y para validación, nunca en cada frame de la UI.
- Caché local de datos orbitales (SQLite/AsyncStorage) con TTL y marca de antigüedad, para operar sin conectividad en el momento del pase.
- Limitador de requests (rate limiter) por proveedor, configurado con los límites de la tabla anterior, y cola de reintentos con backoff exponencial.
- Interfaz común DataProvider (getCatalog, getOrbitalData, getValidationPosition) con implementaciones CelesTrakProvider, SpaceTrackProvider, N2yoProvider y WhereTheIssProvider, y un orquestador que aplica fallback.
- Credenciales (Space-Track, API key de N2YO) fuera del código fuente y del bundle de la app. Recomendado: pequeño backend proxy/caché (Node.js) que guarde las credenciales y reduzca el número de requests; alternativa: almacenamiento seguro con Keychain/Keystore.

4.4 Visualización geográfica (mapa)

- react-native-maps — Marker (posición del satélite), Polyline (ground track) y Circle/Polygon (footprint). Alternativa: MapLibre React Native con tiles propios o de un proveedor; revisar la política de uso de tiles del proveedor elegido.
- Antimeridiano: al construir el ground track, dividir la polilínea en segmentos cuando la longitud salte más de 180° entre muestras consecutivas, para evitar líneas que crucen todo el mapa.
- Rendimiento: limitar los marcadores simultáneos de la capa de constelación (p. ej. subconjunto o agrupamiento), calcular posiciones en lote fuera del hilo de UI y actualizar el estado con throttling.
- Muestreo del ground track: paso de 30-60 s sobre al menos una órbita completa (LEO ≈ 90-100 min), ajustable según el tipo de órbita.

4.5 Sensores y ubicación

- react-native-geolocation-service — posición del observador (lat/lon/alt).
- react-native-sensors o expo-sensors — magnetómetro/giroscopio para el modo brújula/AR simplificado.

Licenciatura en Sistemas de Información

Programación Moviles - 2026

4.6 Sky chart y notificaciones

- react-native-svg — construcción del sky chart polar (círculos de elevación concéntricos, trayectoria del pase, posición actual).
- notifee o react-native-push-notification — recordatorios de pase.
- react-native-background-fetch — recálculo periódico de próximos pases y refresco de datos orbitales para mantener notificaciones vigentes.

5. Arquitectura propuesta

Capa	Responsabilidad	Tecnología
Data Provider Layer	Interfaz común de proveedores (CelesTrak, Space-Track, N2YO, Where the ISS at?), rate limiting, fallback y caché con TTL	fetch + SQLite/AsyncStorage (+ backend proxy opcional)
Orbital Engine	Propagación SGP4, cálculo de posición, punto subsatelital y AOS/LOS	satellite.js (TypeScript wrapper)
Observer Layer	Posición del usuario y orientación del dispositivo	geolocation-service, sensors
Prediction Service	Búsqueda de próximas ventanas de visibilidad por umbral de elevación mínima	TypeScript, Web Worker / JSI si aplica
Geo/Map Layer	Marcador en tiempo real, ground track (con corte en antimeridiano), footprint y capa de constelación	react-native-maps / MapLibre
Validation Layer	Comparación de posiciones calculadas contra N2YO y Where the ISS at?, cálculo del error en km	TypeScript, módulo de pruebas
Presentation Layer	Sky chart, mapa, listado de pases, modo brújula	react-native-svg, Reanimated
Notification Layer	Recordatorios de pase y refresco en background	notifee, background-fetch

Un desafío de diseño central es el costo computacional de la propagación orbital: calcular AOS/LOS de forma naive (evaluando la posición cada pocos segundos durante días) y, además, propagar decenas o cientos de satélites para la capa de constelación es costoso en el hilo de JS. Se recomienda usar un algoritmo de búsqueda por refinamiento (grueso → fino) y, si el rendimiento lo requiere, mover el cálculo a un Web Worker o a JSI/TurboModule para no bloquear la UI.

Licenciatura en Sistemas de Información

Programación Moviles - 2026

6. Entregables

- Código fuente completo en repositorio Git.
- Build de prueba (APK / TestFlight interno) o proyecto ejecutable vía Metro.
- Documento técnico explicando el modelo de propagación orbital utilizado, supuestos y limitaciones (p. ej. no corrección atmosférica fina).
- Informe de validación: tabla de errores de posición (km) entre los cálculos propios y N2YO/Where the ISS at?, con antigüedad de los datos orbitales y conclusiones.
- Documento de decisiones sobre fuentes de datos: justificación del uso del set suplementario de Starlink, límites de cada API y estrategia de caché/fallback.
- Video demo mostrando: selección de un satélite real (ej. ISS), su posición en el mapa en tiempo real con ground track y footprint, la capa de constelación Starlink, el próximo pase visible con sky chart y una notificación disparada.

7. Criterios de evaluación

Criterio	Peso
Correctitud del cálculo orbital y de las geoposiciones, validada contra N2YO/Where the ISS at? con error documentado	25%
Calidad del algoritmo de predicción de ventanas de visibilidad	15%
Visualización geográfica: marcador, ground track (antimeridiano), footprint y capa de constelación	20%
Sky chart y experiencia de uso	10%
Integración de APIs: abstracción de proveedores, caché, rate limiting, fallback y manejo de credenciales	15%
Manejo de background y notificaciones	5%
Calidad de código, arquitectura, rendimiento y documentación	10%

9. Recursos y bibliografía sugerida

- CelesTrak — NORAD Two-Line Element Set Format, documentación y catálogos públicos, incluidos los datos suplementarios (SupGP).
- Vallado, D. — Fundamentals of Astrodynamics and Applications (referencia del algoritmo SGP4/SDP4).
- satellite.js — documentación de la librería (GitHub, shashwatak/satellite-js).
- N2YO API — documentación de endpoints de posición y pases (https://www.n2yo.com/api/), útil como validación cruzada.
- Space-Track.org — API Use Guidelines y límites de uso (< 30 requests/min, 300/hora).

Licenciatura en Sistemas de Información

Programación Moviles - 2026

- Where the ISS at? — API REST de la ISS (https://wheretheiss.at/w/developer).
- Starlink — documentación de APIs Enterprise y Space Traffic Coordination API (para comprender sus alcances y restricciones de acceso).
- ITU-R — recomendaciones sobre elevación mínima de operación para enlaces satelitales (contexto de telecomunicaciones satelitales).